

HE-Guardrail: A Homomorphic Guardrail Against Jailbreak Attacks for Encrypted Large Language Model Inference

Byeongseo Min*, Yongwoo Lee[†], Young-Sik Kim[‡], and Yongjune Kim^{*§}

^{*}{minbyeongseo, yongjune}@postech.ac.kr, Pohang University of Science and Technology (POSTECH)

[†]yongwoo@inha.ac.kr, Inha University

[‡]ysk@dgist.ac.kr, Daegu Gyeongbuk Institute of Science and Technology (DGIST)

Abstract—Homomorphic encryption (HE) has emerged as a promising approach to privacy-preserving machine learning (PPML), enabling computation directly over encrypted data. In HE-based PPML, a client submits an encrypted input to the server, which evaluates models such as large language models (LLMs) without access to the underlying plaintext. However, we identify a critical security vulnerability in this setting: HE-LLM inference is vulnerable to malicious clients that submit adversarial prompts, such as *jailbreak attacks*. The same confidentiality that protects benign clients also prevents the server from inspecting incoming prompts or generated responses, making adversarial attempts difficult to detect or block and potentially allowing successful attacks to remain entirely invisible to the server. To address this vulnerability, we propose HE-GUARDRAIL, a framework that evaluates guardrail mechanisms entirely over encrypted data and homomorphically controls whether the target-model response is returned to the client. We instantiate HE-Guardrail with three representative guardrails—Llama Guard, JBSHield, and GradSafe. Our results show that HE-Guardrail closely reproduces the decisions of the corresponding plaintext guardrails in the encrypted domain, with distinct security–efficiency–utility trade-offs.

1. Introduction

The growing deployment of artificial intelligence (AI) services has heightened concerns about privacy and security. In particular, machine learning often involves privacy-sensitive data during training and inference, motivating extensive research on privacy-preserving machine learning (PPML) [1], [2], [3]. Representative approaches include differential privacy (DP) [4], secure multi-party computation (SMPC) [5], [6], and homomorphic encryption (HE) [7], [8].

Among these approaches, HE is a promising direction for privacy-preserving inference as it provides strong cryptographic confidentiality guarantees while enabling computation directly over encrypted data. In a typical HE-based PPML pipeline, a client sends an encrypted input to the server, which performs inference using a homomorphically implemented model and returns the encrypted result. Any

plaintext input or output of the model remains hidden from the server throughout the process. Early studies on homomorphic Transformer inference primarily focused on encoder-based architectures such as BERT [9], [10], [11], while recent advances have extended encrypted inference to large language models (LLMs) such as Llama or GPT [12], [13], [14].

Existing HE-based PPML systems commonly adopt a *semi-honest* (a.k.a. *honest-but-curious*) server as their primary threat model [3], while malicious behavior from the client is often outside the scope of consideration. In this work, we make a critical observation for HE-LLM inference: such systems are highly vulnerable to malicious clients that submit adversarial prompts, such as *jailbreak attacks*. Jailbreak attacks [15], [16] are designed to bypass the safety alignment of LLMs and elicit harmful or otherwise restricted responses. Because the prompt and model computation remain encrypted, the server cannot directly inspect such adversarial inputs and may remain unaware even when an attack successfully elicits a harmful response. Consequently, adversarial interactions can remain hidden throughout the encrypted inference pipeline.

A common approach to mitigating jailbreak attacks is to employ *guardrails* [17], [18], which act as external defense mechanisms that monitor and control LLM interactions. We refer to the recent Systematization of Knowledge (SoK) study [19] for a comprehensive taxonomy and evaluation framework for existing jailbreak guardrails. Based on their intervention stage, guardrails can be categorized as pre-processing, intra-processing, or post-processing. Pre-processing guardrails [18], [20] examine the input prompt, intra-processing guardrails [21], [22] analyze the target model’s internal states or gradients, and post-processing guardrails [23], [24] inspect the generated output. However, these guardrails are designed for plaintext inference and cannot be directly applied when the information required for their safety decisions remains encrypted.

To address this challenge, we propose **HE-Guardrail**, a framework that implements guardrails entirely under HE and integrates their encrypted safety decisions into HE-LLM inference. Figure 1 contrasts the HE-Guardrail framework with the conventional HE-LLM pipeline. In the conventional pipeline (top), the server cannot inspect the encrypted jailbreak prompt and thus has no choice but to return the

[§]Corresponding author.

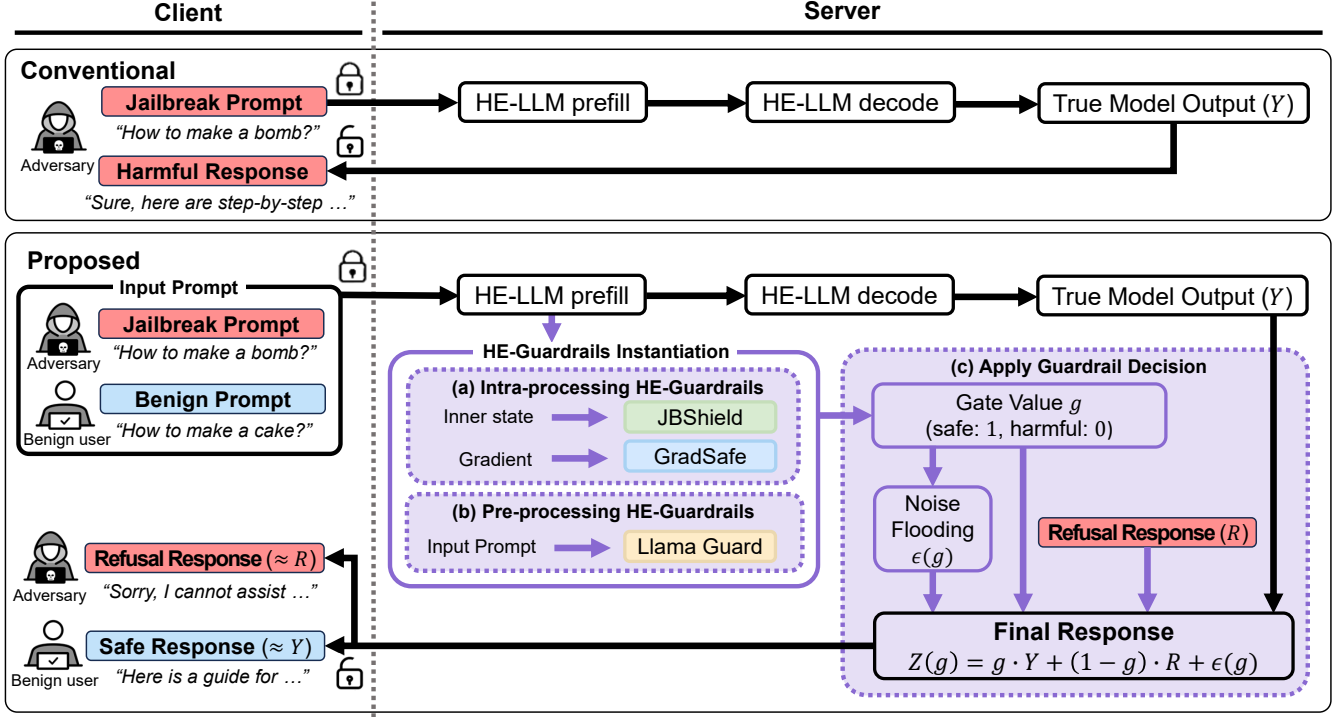


Figure 1. Comparison of conventional HE-LLM inference and the proposed HE-GUARDRAIL pipeline, which performs encrypted jailbreak detection and selectively returns either the target-model output or a refusal response.

resulting harmful response to the client. In our proposed HE-Guardrail framework (bottom), both guardrail evaluation and response control are performed over encrypted data, without revealing the prompt, model response, or guardrail decision to the server.

As shown in Figure 1, we instantiate HE-Guardrail with (a) intra-processing guardrails, which take encrypted internal signals of the target model, and (b) a pre-processing guardrail, which takes the encrypted input prompt. As shown in (c), the resulting encrypted guardrail decision g (1: safe, 0: harmful) selects between the target-model response Y and a predefined refusal response R :

$$Z(g) = g \cdot Y + (1 - g) \cdot R + \epsilon(g), \quad (1)$$

where the *gated noise flooding* term $\epsilon(g)$ prevents a malicious client from recovering the suppressed response. The resulting ciphertext therefore reveals only the response permitted by the encrypted guardrail decision while preserving the confidentiality of the underlying HE inference pipeline.

Our contributions are summarized as follows:

- **HE-Guardrail Framework:** We introduce HE-Guardrail, a general framework for evaluating LLM guardrails entirely under HE and enforcing their encrypted safety decisions through homomorphic response gating. We further introduce the *gated noise flooding* term $\epsilon(g)$ to prevent recovery of the suppressed target-model response from the gated output.

- **Intra-processing HE-Guardrails:** We develop HE-compatible realizations of JBSHield [22] and GradSafe [21] by exploiting encrypted information generated within the target-model inference. In particular, we implement *encrypted gradient evaluation* for GradSafe, while JBSHield directly leverages encrypted internal model states. These designs demonstrate that intra-processing guardrails are a promising direction for HE-LLM inference, where the server can reuse the target model’s encrypted computation without exposing its internal signals.
- **Pre-processing HE-Guardrails:** We develop HE-compatible realizations of Llama Guard [18] that evaluate encrypted input prompts. To efficiently execute multiple models with different weights, we extend interleaved batching [13] to *multi-model serving*, which assigns different model weights to each interleaved batch within a single ciphertext. This enables the target model and guardrail models to be evaluated concurrently within a unified HE inference pipeline.

We evaluate the three HE-Guardrail instantiations under the encrypted domain from security, efficiency, and utility perspectives. For security, we measure jailbreak attack success rate and pass guardrail rate; for efficiency, we measure memory usage and latency overhead; and for utility, we evaluate benign-prompt behavior. Across the evaluated settings, HE-Guardrail achieves jailbreak defense performance close

to the corresponding plaintext guardrails, while exhibiting distinct security–efficiency–utility trade-offs across different guardrail designs.

2. Preliminaries

2.1. Homomorphic Encryption

Homomorphic encryption (HE) [25], [26] enables computation directly over encrypted data without revealing the underlying plaintext. In this work, we use the RNS variant of CKKS, an HE scheme supporting arithmetic over real or complex numbers [25], [27]. For a ring degree N , CKKS packs up to $N/2$ values into a ciphertext and processes them in a single-instruction-multiple-data (SIMD) manner. We refer to the arrangement of values across these slots as *ciphertext packing*. For a ciphertext ct_i , we use $ct_i[j]$ to denote the encrypted value in its j -th slot.

2.1.1. HE Operations. RNS-CKKS mainly supports homomorphic addition (Add), ciphertext–plaintext multiplication (Mult_p), ciphertext–ciphertext multiplication (Mult_c), and cyclic slot rotation (Rot). Add, Mult_p , and Mult_c perform element-wise arithmetic over corresponding slots, while $\text{Rot}(ct, r)$ cyclically rotates the slots of ct by r positions. Multiplication followed by rescaling typically consumes a *level* from the modulus chain, and the maximum number of consecutive level-consuming operations determines the multiplicative depth. Once the available levels are exhausted, bootstrapping refreshes a ciphertext at substantial computational cost.

2.1.2. Security. RNS-CKKS is known to satisfy indistinguishability under chosen-plaintext attack (IND-CPA) security [28]. Hence, a server observing only ciphertexts and their homomorphic evaluations cannot computationally distinguish the underlying plaintexts or infer their contents. Consequently, plaintext confidentiality is preserved throughout server-side evaluation.

2.1.3. HE-based PPML. HE-based PPML systems have introduced contributions along several major design dimensions, including the target model architecture, ciphertext packing layout, homomorphic matrix-multiplication algorithm, and implementation of nonlinear operations. While linear layers can be evaluated using the arithmetic operations natively supported by HE, nonlinear functions are generally implemented through HE-friendly replacements or polynomial approximations.

Early HE-based PPML primarily focused on relatively small neural networks such as CNNs [8], [29]. With the emergence of Transformer architectures, subsequent work extended encrypted inference to BERT-style models [9], [10], [13], [30], [31], [32]. From the perspective of ciphertext packing and data layout, representative approaches include column-oriented packing in Powerformer [10], diagonal packing in THOR [9], and tricyclic encodings in Tricycle [31]. MOAI [13] uniquely introduces interleaved

batching to efficiently process multiple inputs sharing the same model weights.

HE-LLM. Recent work has extended HE inference to generative LLMs, first focusing on the *prefill* stage for Llama [33], [34], [35], [36] and GPT [12], [34], [35], [36]. Subsequent work further supports autoregressive *decoding* [14], [36], which additionally requires efficient management of encrypted KV caches. In autoregressive decoding, the selected next token can be represented as a one-hot vector and multiplied by the embedding table to obtain the input embedding for the next decoding step. MOAI [13] also demonstrates encrypted prefill for Llama-3-8B and shows that its framework can be extended to autoregressive decoding, with explicitly specified ciphertext packing layouts and an open-source implementation.

2.1.4. Threat Model in HE-based PPML. HE-based PPML commonly considers a semi-honest server that follows the prescribed inference protocol but attempts to learn information about the client’s private input [3]. Some secure inference frameworks [14], [37], [38] additionally assume a semi-honest client, with both parties following the protocol while attempting to learn additional information; in such settings, the goal is to protect both the client’s private input and the server’s proprietary model.

2.2. Jailbreak Attacks and Guardrails

Below, we summarize the major categories of jailbreak attacks and guardrails following the taxonomy of Wang et al. [19].

2.2.1. Jailbreak Attacks. Jailbreak attacks aim to bypass the safety alignment of an LLM and induce harmful or otherwise restricted responses. Following Wang et al. [19], jailbreak attacks can broadly be divided into single-turn and multi-turn attacks. Single-turn attacks attempt to elicit a harmful response within a single interaction, whereas multi-turn attacks develop or distribute the malicious objective across multiple interactions. Single-turn attacks include *manual* attacks [39] based on handcrafted jailbreak prompts, *optimization-based* attacks [15], [16] that search for adversarial prompts, *generation-based* attacks [40], [41] that use auxiliary models to generate or refine jailbreak prompts, and *implicit* attacks [42], [43] that conceal harmful intent through transformations or indirect instructions. Multi-turn attacks [44], [45] instead distribute or progressively develop the malicious objective across multiple interactions. Multi-turn attacks remain particularly challenging for existing guardrails, highlighting the need for more robust defenses specifically designed to reason over evolving multi-turn interactions.

2.2.2. Jailbreak Guardrails. Jailbreak guardrails detect and block unsafe interactions without modifying the target model itself. Based on the intervention stage, guardrails can be categorized into *pre-processing*, *intra-processing*, and *post-processing* methods [19].

Pre-Processing Guardrails. Pre-processing guardrails inspect the user prompt. Representative methods such as Llama Guard [18], SelfDefend [20] and GuardReasoner [46] use a separate language model to classify the safety of an incoming request. They are applicable even to black-box target models and can reject malicious requests before incurring target-model generation cost, but their decisions are limited to information available from the input.

Intra-Processing Guardrails. Intra-processing guardrails exploit internal signals generated during target-model inference, such as hidden representations or gradients. For example, GradSafe [21] uses gradient information, while JB-Shield [22] detects jailbreaks from internal model representations. Such methods can provide deeper and more nuanced insights into the target model’s behavior and vulnerabilities, but require white-box access to the target model.

Post-Processing Guardrails. Post-processing guardrails [23], [24] inspect the generated response before returning it to the user. They can directly detect harmful outputs even when the corresponding input appears benign, but require the target model to complete generation before the safety decision is made. Consequently, they generally incur the full generation cost and additional end-to-end latency.

3. HE-Guardrail

In this section, we present **HE-Guardrail**, our framework for integrating jailbreak guardrails into HE-LLM inference. We first define the threat model, target model, and overall framework, and then instantiate HE-Guardrail with one pre-processing and two intra-processing guardrails. We exclude post-processing guardrails because they can begin only after response generation and therefore directly add to the end-to-end latency, a critical bottleneck in HE inference. In contrast, our pre-processing guardrails can run concurrently with the target model, while the intra-processing guardrails can be evaluated using information obtained during or immediately after prefill, independently of subsequent autoregressive decoding. Consequently, these designs limit the additional latency introduced on the target model’s critical path.

3.1. Threat Model and Target Model

We consider a two-party HE-LLM inference setting consisting of a client $\mathcal{C}$ and a server $\mathcal{S}$. The server holds the target LLM and guardrail models, while the client holds a private input prompt and the HE secret key. The client encrypts its input and sends the resulting ciphertexts to the server, which evaluates the prescribed inference pipeline homomorphically and returns only the encrypted final response.

Semi-Honest Server. Following the standard threat model in HE-based PPML, we assume that the server is semi-honest: it correctly follows the prescribed protocol but may attempt to infer information about the client’s private input from the ciphertexts and intermediate computations. The server does not possess the secret key and therefore cannot directly

inspect the plaintext prompt, intermediate representations, or generated response.

Malicious Client. Unlike conventional HE-based PPML, we allow the client to be malicious in its choice of input. Specifically, the client may intentionally submit adversarial prompts, including jailbreak prompts, with the goal of eliciting a harmful response from the target LLM. We assume that the client follows the underlying HE protocol correctly, but places no restriction on the plaintext input that it encrypts. After inference, the client decrypts the returned ciphertext and may attempt to recover the target-model response even when the guardrail blocks the request.

Target Model. We adopt Llama-3-8B-Instruct [47] as our target model, following the main target model used in Wang et al. [19].

3.2. Framework

HE-Guardrail integrates jailbreak guardrails into HE-LLM inference and enforces their safety decisions entirely over encrypted data. As illustrated in Figure 1, the server evaluates a guardrail using either the encrypted input prompt or encrypted internal signals from the target model, and obtains an encrypted binary decision g , where $g = 1$ indicates acceptance and $g = 0$ indicates rejection. Since g remains encrypted, the server applies this decision to the returned response through homomorphic arithmetic without observing it.

Guardrails typically reach their decision by comparing a detection score with a threshold. We therefore abstract each decision criterion by an encrypted score p and a threshold δ , oriented such that $p > \delta$ indicates a benign interaction. Since comparison is not natively supported in CKKS, the server homomorphically computes the gate as

$$g = \frac{1 + \text{sgn}_{\text{poly}}(p - \delta)}{2}, \quad (2)$$

where sgn_{poly} is a polynomial approximation to the sign function. We implement sgn_{poly} using the minimax composite-polynomial method of Lee et al. [48], with component degrees (15, 27, 29). Accordingly, g approaches 1 when $p > \delta$ and 0 when $p < \delta$, corresponding to acceptance and rejection, respectively. For readability, we describe the following operations using their underlying values, although p , g , and the target-model outputs remain encrypted throughout server-side evaluation.

At autoregressive decoding step t , let y_t denote the token selected by the target LLM and

$$Y_t = \mathbf{e}_{y_t} \in \{0, 1\}^V \quad (3)$$

its one-hot representation, where V is the vocabulary size. The next-step input embedding is obtained as EY_t using the embedding table $E \in \mathbb{R}^{d \times V}$. For the refusal path, the server prepares a predefined refusal response R in advance, such as “Sorry, I cannot assist with that request.” Similarly, let $R_t \in \{0, 1\}^V$ denote the one-hot representation of its t -th token in the target model’s vocabulary. Since the refusal response

is usually shorter than the generation budget, the remaining positions are filled with the end-of-sequence (EOS) token.

Given the encrypted gate g , HE-Guardrail homomorphically constructs the client-facing response as

$$Z_t(g) = gY_t + (1 - g)R_t + \epsilon_t(g), \quad (4)$$

where the first two terms interpolate between the target-model token Y_t and the predefined refusal token R_t according to the encrypted gate. The term $\epsilon_t(g)$ denotes the gated noise flooding term used to mask residual information from the suppressed response. We define $\epsilon_t(g)$ in the following subsection.

3.2.1. Gated Noise Flooding. In practice, the gate g may not be exactly 0 or 1, for example, due to the approximation error of sgn_{poly} in Eq. (2) or the inherent noise of CKKS. Consequently, even a rejected request may result in a small nonzero gate g , leaving residual information from the target-model output. For example, consider $Y_t = (1, 0, 0, 0)^\top$, $R_t = (0, 0, 0, 1)^\top$, and $g = 0.01$. Without the noise term in Eq. (4), the returned vector becomes

$$gY_t + (1 - g)R_t = (0.01, 0, 0, 0.99)^\top. \quad (5)$$

After decryption, the client obtains the refusal token through plaintext argmax decoding, but the residual value 0.01 still reveals the coordinate of the suppressed target token.

The gated noise flooding term $\epsilon_t(g)$ masks this residual leakage. We construct it at a scale proportional to $1 - g$ as

$$\epsilon_t(g) = (1 - g)\xi_t, \quad (6)$$

where ξ_t is a small random noise vector freshly sampled by the server for each output and kept private from the client. The noise is therefore activated for rejected requests with $g \approx 0$ and suppressed for accepted requests with $g \approx 1$. In the above example, sampling $\xi_t = (0.12, -0.08, 0.15, -0.05)^\top$ yields

$$Z_t(g) \approx (0.13, -0.08, 0.15, 0.94)^\top. \quad (7)$$

After decrypting $Z_t(g)$, the client still obtains the refusal token ID through argmax decoding, but can no longer identify the suppressed target-model token, since its residual 0.01 is buried in noise of larger magnitude.

3.3. Intra-processing HE-Guardrails

Key Observation. Intra-processing guardrails are a particularly promising design direction for HE-Guardrail. In plaintext deployments, intra-processing guardrails generally require white-box access to the target model, which limits their applicability when the model is available only through a black-box API. This white-box requirement has also been identified as a major applicability limitation of intra-processing guardrails in prior systematic evaluations [19]. HE-based inference fundamentally changes this setting. The server typically owns and homomorphically evaluates the target LLM and therefore has algorithmic access to its model structure and encrypted internal signals, even though their

plaintext values remain hidden. It can thus directly use encrypted hidden states or homomorphically derive gradients required by intra-processing guardrails. Consequently, the white-box requirement that limits these methods in plaintext deployments becomes substantially less restrictive in HE-based inference.

We instantiate this direction with two representative guardrails that exploit different internal signals: JBSHield, which detects jailbreaks from internal model representations, and GradSafe, which uses gradient information obtained during prefill.

3.3.1. JBSHield. In plaintext JBSHield-D [22], the toxic concept is calibrated from representation differences between harmful and benign prompts, whereas the jailbreak concept is calibrated from differences between jailbreak and harmful prompts. For a test prompt, JBSHield-D similarly constructs a toxic direction relative to a benign anchor representation and a jailbreak direction relative to a harmful anchor representation. It then computes the *cosine similarities* of these directions with the corresponding calibrated toxic and jailbreak concept vectors. The input is classified as a jailbreak when both similarities exceed their respective thresholds.

Let $H^{(\ell)}(x) \in \mathbb{R}^{T \times d}$ denote the hidden representation after the ℓ -th transformer layer for a prompt x with T tokens. Following JBSHield-D [22], we use the last-token representation

$$\mathbf{h}^{(\ell)}(x) = H^{(\ell)}(x)[T - 1, :] \in \mathbb{R}^d \quad (8)$$

as the sentence representation of x . In our Llama-3-8B-Instruct implementation, we fix $\ell = 32$ and hence use $\mathbf{h}(x) = H^{(32)}(x)[T - 1, :] \in \mathbb{R}^{4096}$. We adapt the subsequent concept extraction and similarity evaluation to HE as summarized in Algorithm 1.

Offline Calibration. Following JBSHield-D, the server constructs a benign anchor representation μ_b , a harmful anchor representation μ_h , and the toxic and jailbreak concept vectors $\mathbf{s}_t$ and $\mathbf{s}_j$ from calibration prompts. The toxic concept is extracted from representation differences between harmful and benign prompts, while the jailbreak concept is extracted from differences between jailbreak and harmful prompts, using rank-one SVD as in the original method [22]. Since these quantities depend only on the server-owned model and calibration data, all calibration is performed in plaintext before deployment. We additionally pre-normalize $\mathbf{s}_t$ and $\mathbf{s}_j$ to avoid evaluating their norms homomorphically.

SVD Simplification. In the original online detection [22], JBSHield-D forms a single-row difference matrix from the test representation and applies rank-one SVD before cosine-similarity evaluation. For example, the toxic concept is constructed from

$$D_t = [\mathbf{h}(x) - \mu_b] = [\mathbf{c}_t]. \quad (9)$$

Since D_t contains only one difference vector, its right singular direction is $\mathbf{c}_t / \|\mathbf{c}_t\|_2$, up to sign. Under the same orientation convention as the plaintext implementation, the

Algorithm 1 HE-JBShield-D

Require: Benign, harmful, and jailbreak calibration prompts; encrypted input prompt x

Ensure: Encrypted acceptance gate g_{JB}

Offline phase

- 1: Compute benign and harmful anchor representations μ_b and μ_h
- 2: Extract toxic and jailbreak concept vectors s_t and s_j using rank-one SVD
- 3: Normalize $\hat{s}_k \leftarrow s_k / \|s_k\|_2$ for $k \in \{t, j\}$
- 4: Set $\theta_t \leftarrow 0.037$ and $\theta_j \leftarrow 0$

Online phase

- 5: Obtain encrypted $\mathbf{h}(x)$ from the target-model prefill
 - 6: $\mathbf{c}_t \leftarrow \mathbf{h}(x) - \mu_b$
 - 7: $\mathbf{c}_j \leftarrow \mathbf{h}(x) - \mu_h$
 - 8: **for** $k \in \{t, j\}$ **do**
 - 9: $q_k \leftarrow \sum_i \mathbf{c}_k[i]^2$
 - 10: $\nu_k \leftarrow \text{InvSqrt}_{\text{HE}}(q_k)$
 - 11: $\sigma_k \leftarrow \nu_k \sum_i \mathbf{c}_k[i] \hat{s}_k[i]$
 - 12: $d_k \leftarrow (1 + \text{sgn}_{\text{HE}}(\sigma_k - \theta_k)) / 2$
 - 13: **end for**
 - 14: $g_{JB} \leftarrow 1 - d_t d_j$
 - 15: **return** g_{JB}
-

subsequent cosine similarity is therefore identical to directly computing

$$\cos(\mathbf{c}_t, \mathbf{s}_t). \quad (10)$$

We consequently eliminate the online SVD and directly use $\mathbf{c}_t$ and $\mathbf{c}_j$ for toxic and jailbreak concept detection. The offline SVD is retained because it extracts the principal concept directions from multiple calibration samples.

Homomorphic Cosine Similarity. For $k \in \{t, j\}$, the required similarity is

$$\sigma_k = \frac{\langle \mathbf{c}_k, \mathbf{s}_k \rangle}{\|\mathbf{c}_k\|_2 \|\mathbf{s}_k\|_2} = \langle \mathbf{c}_k, \hat{\mathbf{s}}_k \rangle \frac{1}{\sqrt{\sum_i \mathbf{c}_k[i]^2}}. \quad (11)$$

The squared norm is evaluated by element-wise multiplication followed by slot summation. Since inverse square root is not natively supported by CKKS, we evaluate $1/\sqrt{q_k}$ using Goldschmidt iterations. The remaining operations in Eq. (11) consist only of multiplication and slot summation.

Finally, the toxic and jailbreak activations are obtained by comparing σ_t and σ_j with $\theta_t = 0.037$ and $\theta_j = 0$, respectively, using the same sgn_{poly} operation as in Section 3.2. The values d_t and d_j approach one when their corresponding concepts are activated. Since JBShield-D identifies a jailbreak only when both concepts are activated, we directly produce the acceptance gate

$$g_{JB} = 1 - d_t d_j. \quad (12)$$

3.3.2. GradSafe. GradSafe [21] detects unsafe prompts by analyzing gradients of safety-critical parameters in the target LLM. Its key observation is that unsafe prompts paired with a fixed compliance response induce similar gradient directions on a small subset of model parameters, whereas safe

prompts exhibit substantially different directions. GradSafe first identifies these safety-critical parameters and constructs corresponding unsafe gradient references in an offline phase. For a new prompt, it then computes the same gradients and compares them with the references using cosine similarity.

In our implementation, we consider row- and column-wise gradient slices of the attention projection weights $\{\mathbf{W}_q^{(\ell)}, \mathbf{W}_k^{(\ell)}, \mathbf{W}_v^{(\ell)}, \mathbf{W}_o^{(\ell)}\}$ and the MLP projection weights $\{\mathbf{W}_{\text{gate}}^{(\ell)}, \mathbf{W}_{\text{up}}^{(\ell)}, \mathbf{W}_{\text{down}}^{(\ell)}\}$, where $\ell = 1, \dots, L$ denotes the transformer layer. Let $\mathcal{J}$ denote the set of candidate gradient slices from these projection weights. For a prompt x paired with a fixed compliance response r_c , let $\mathbf{g}_j(x)$ denote the gradient vector corresponding to slice $j \in \mathcal{J}$ of the causal-LM loss $\mathcal{L}(x, r_c)$.

Let $\mathcal{X}_u$ and $\mathcal{X}_s$ denote the unsafe and safe reference prompt sets used during offline calibration. Following GradSafe-Zero, we use a similarity-gap threshold of 1 to identify safety-critical slices and an online detection threshold of 0.25. Algorithm 2 summarizes our HE implementation.

Gradient-Only HE Backpropagation. HE-based training [11], [49] is challenging because weight updates turn plaintext model parameters into ciphertexts, replacing efficient ciphertext–plaintext operations with more expensive ciphertext–ciphertext operations, while approximation errors may accumulate across optimization steps. In contrast, GradSafe requires only gradient extraction without weight updates, making HE backpropagation substantially more practical.

Offline Calibration. For each candidate slice $j \in \mathcal{J}$, GradSafe first constructs an unsafe gradient reference

$$\mathbf{r}_j = \frac{1}{|\mathcal{X}_u|} \sum_{x \in \mathcal{X}_u} \mathbf{g}_j(x). \quad (13)$$

The safety-criticality of the slice is then determined by the difference between its average cosine similarity for unsafe and safe reference prompts,

$$\Delta_j = \frac{1}{|\mathcal{X}_u|} \sum_{x \in \mathcal{X}_u} \cos(\mathbf{g}_j(x), \mathbf{r}_j) - \frac{1}{|\mathcal{X}_s|} \sum_{x \in \mathcal{X}_s} \cos(\mathbf{g}_j(x), \mathbf{r}_j). \quad (14)$$

We retain

$$\mathcal{J}^* = \{j \in \mathcal{J} : \Delta_j > \lambda_{\text{gap}}\} \quad (15)$$

as the safety-critical slice set, where we set $\lambda_{\text{gap}} = 1$ following GradSafe-Zero. Because this phase uses only the server-owned model and reference data, it is performed entirely in plaintext. We also precompute

$$\hat{\mathbf{r}}_j = \frac{\mathbf{r}_j}{\|\mathbf{r}_j\|_2}, \quad j \in \mathcal{J}^*, \quad (16)$$

so that encrypted cosine-similarity evaluation requires homomorphic normalization only for the online gradient $\mathbf{g}_j(x)$.

Backward Initialization. After the encrypted prompt prefill, the resulting state is shared by the target-model decoding and GradSafe detection. GradSafe constructs the gradient of a causal-LM objective toward a fixed compliance response

Algorithm 2 HE-GradSafe

Require: Reference sets $\mathcal{X}_u$ and $\mathcal{X}_s$, encrypted prompt x , compliance response r_c

Ensure: Encrypted acceptance gate g_{GS}

Offline phase

```
1: for each candidate slice  $j \in \mathcal{J}$  do
2:   Compute  $\mathbf{g}_j(x')$  for all  $x' \in \mathcal{X}_u \cup \mathcal{X}_s$ 
3:    $\mathbf{r}_j \leftarrow \frac{1}{|\mathcal{X}_u|} \sum_{x' \in \mathcal{X}_u} \mathbf{g}_j(x')$ 
4:   Compute similarity gap  $\Delta_j$  between unsafe and safe references
5: end for
6:  $\mathcal{J}^* \leftarrow \{j \in \mathcal{J} : \Delta_j > \lambda_{\text{gap}}\}$ 
7: for each  $j \in \mathcal{J}^*$  do
8:    $\hat{\mathbf{r}}_j \leftarrow \mathbf{r}_j / \|\mathbf{r}_j\|_2$ 
9: end for
Online phase
10: Run encrypted forward pass for  $(x, r_c)$  and store required checkpoints at a low ciphertext level
11: Initialize the backward gradient from the causal-LM objective
12:  $A \leftarrow 0$ 
13: for each operation in reverse order do
14:   if operation is Softmax then
15:     Backpropagate using the checkpointed softmax output
16:   else if operation is RMSNorm then
17:     Backpropagate using the checkpointed input and inverse RMS
18:   else if operation is SiLU then
19:     Backpropagate using the checkpointed sigmoid and SiLU outputs
20:   else
21:     Backpropagate using the corresponding HE linear/matrix-multiplication operation
22:   end if
23:   if a safety-critical gradient slice  $j \in \mathcal{J}^*$  is produced then
24:      $q_j \leftarrow \sum_i \mathbf{g}_j[i]^2$ 
25:      $\nu_j \leftarrow \text{InvSqrt}_{\text{HE}}(q_j)$ 
26:      $\gamma_j \leftarrow \nu_j \sum_i \mathbf{g}_j[i] \hat{\mathbf{r}}_j[i]$ 
27:      $A \leftarrow A + \gamma_j$ 
28:     Discard  $\mathbf{g}_j$ 
29:   end if
30: end for
31:  $s_{GS} \leftarrow A / |\mathcal{J}^*|$ 
32:  $d_{GS} \leftarrow (1 + \text{sgn}_{\text{HE}}(s_{GS} - 0.25)) / 2$ 
33:  $g_{GS} \leftarrow 1 - d_{GS}$ 
34: return  $g_{GS}$ 
```

r_c , without modifying the actual decoding sequence. For a single response token, the backward pass can be initialized directly from the prefill logits as

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{e}_{r_c}, \quad (17)$$

where $\mathbf{p}$ is the softmax output at the last prompt position and $\mathbf{e}_{r_c}$ is the one-hot representation of the compliance token.

The resulting backward pass can proceed independently of subsequent autoregressive decoding.

Low-Level Checkpointing. Efficient backpropagation requires intermediate values from the forward pass. Keeping all such ciphertexts at their original levels, however, incurs substantial memory consumption because high-level CKKS ciphertexts contain more RNS limbs. We therefore level-drop the required checkpoints to a designated low storage level immediately after their last forward use. When a checkpoint is required during backward propagation, we bootstrap it and subsequently level-align it with the current backward gradient ciphertext before evaluation. This strategy trades additional bootstrapping for substantially lower checkpoint memory consumption.

Checkpointed Nonlinear Derivatives. The nonlinear operations required in the transformer backward pass can be efficiently evaluated by reusing quantities already computed during the forward pass. For a softmax input $\mathbf{a}$, let $\mathbf{p} = \text{Softmax}_{\text{HE}}(\mathbf{a})$ denote its checkpointed output, and let $\bar{\mathbf{p}}$ denote the incoming gradient during backward propagation. The gradient with respect to $\mathbf{a}$ is

$$\bar{\mathbf{a}} = \mathbf{p} \odot (\bar{\mathbf{p}} - \langle \bar{\mathbf{p}}, \mathbf{p} \rangle \mathbf{1}). \quad (18)$$

Thus, no additional exponential evaluation is required during the softmax backward pass.

For RMSNorm, let $\mathbf{x} \in \mathbb{R}^d$ denote its input and define the inverse RMS value

$$r = \left(\frac{1}{d} \|\mathbf{x}\|_2^2 + \varepsilon_{\text{rms}} \right)^{-1/2}, \quad (19)$$

where ε_{rms} is the numerical-stability constant of RMSNorm. Given the RMSNorm weight $\mathbf{w}$ and the incoming gradient $\bar{\mathbf{y}}$ with respect to its output $\mathbf{y} = r \mathbf{w} \odot \mathbf{x}$, define $\mathbf{u} = \bar{\mathbf{y}} \odot \mathbf{w}$. Using the checkpointed $\mathbf{x}$ and r , the input gradient is

$$\bar{\mathbf{x}} = r \mathbf{u} - \frac{r^3}{d} \mathbf{x} \langle \mathbf{u}, \mathbf{x} \rangle. \quad (20)$$

Hence, the inverse square root already evaluated during the forward pass can be reused without another $\text{InvSqrt}_{\text{HE}}$ evaluation.

For SiLU, let

$$y = u s, \quad s = \text{Sigmoid}_{\text{HE}}(u), \quad (21)$$

where s and y are checkpointed during the forward pass. For an incoming gradient $\bar{y}$, the input gradient is

$$\bar{u} = \bar{y} [s + y(1 - s)]. \quad (22)$$

Therefore, the sigmoid approximation does not need to be reevaluated during backward propagation.

Streaming Gradient Similarity. To reduce memory usage, we do not store all safety-critical gradients. Instead, each gradient slice is immediately compared with its pre-normalized unsafe reference using encrypted cosine similarity, accumulated into a running score, and then discarded. GradSafe-Zero averages the accumulated similarities and applies the online threshold 0.25 through sgn_{HE} to obtain the final acceptance gate.

3.4. Pre-Processing HE-Guardrails

Pre-processing guardrails evaluate the prompt independently of target response generation. We instantiate this approach with Llama Guard 3-8B [47]. Let x be the user prompt and $\mathcal{T}_T$, $\mathcal{T}_{LG}$ be the target and guardrail prompt templates. The model inputs are $x_T = \mathcal{T}_T(x)$ and $x_{LG} = \mathcal{T}_{LG}(x)$; the latter includes the guardrail’s safety-assessment instructions.

For $M \in \{T, LG\}$, let n_M be the token count and t_i^M the i -th token ID after formatting and tokenization. The corresponding one-hot input and embedding are

$$\mathbf{o}_i^M = \mathbf{e}_{t_i^M} \in \{0, 1\}^{V_M}, \quad \mathbf{x}_i^M = E_M \mathbf{o}_i^M, \quad (23)$$

where $E_M \in \mathbb{R}^{d_M \times V_M}$ is model M ’s plaintext embedding table. The two models use their own weights and embedding tables, while the user-dependent one-hot inputs and embeddings remain encrypted. The formatted streams are padded to a common length with their respective attention masks.

3.4.1. Multi-Model Serving. Llama Guard requires an additional HE-LLM evaluation beyond the target model. Although the two models can run concurrently, executing them as independent HE pipelines doubles the ciphertext memory consumption. To avoid this overhead, we extend MOAI’s interleaved batching [13] by assigning model-specific plaintext weights to each interleaved lane instead of repeating one model’s weights across all lanes, so that the target and guardrail models are evaluated within a single set of ciphertexts.

For lane $b \in \{0, \dots, B-1\}$, let $X^{(b)} \in \mathbb{R}^{n \times d}$ be its activation and $W^{(b)} \in \mathbb{R}^{d \times d'}$ its weight matrix. For ciphertexts under a common HE key, the interleaved assignments are

$$\begin{aligned} \text{ct}_i[Br + b] &= X^{(b)}[r, i], \\ \text{pt}_{i,j}[Br + b] &= W^{(b)}[i, j], \end{aligned} \quad (24)$$

for $0 \leq r < n$, $0 \leq i < d$, and $0 \leq j < d'$. The packed projection is

$$\text{ct}_j^{\text{out}} = \sum_{i=0}^{d-1} \text{Mult}_p(\text{ct}_i, \text{pt}_{i,j}), \quad (25)$$

whose slot $Br + b$ represents $(X^{(b)}W^{(b)})[r, j]$.

3.4.2. Llama Guard Decision. Let $\mathbf{z}_{LG} \in \mathbb{R}^{V_{LG}}$ be Llama Guard’s logits at the first safety-decision position, and let t_{safe} and t_{unsafe} be the corresponding token IDs. Our implementation blocks when $\mathbf{z}_{LG}[t_{\text{unsafe}}] > \mathbf{z}_{LG}[t_{\text{safe}}]$. We therefore compute the encrypted acceptance margin and gate directly as

$$\begin{aligned} m_{LG} &= \mathbf{z}_{LG}[t_{\text{safe}}] - \mathbf{z}_{LG}[t_{\text{unsafe}}], \\ g_{LG} &= \frac{1 + \text{sgn}_{\text{HE}}(m_{LG})}{2}. \end{aligned} \quad (26)$$

This comparison does not require a softmax over the vocabulary or generation of a textual explanation. It is equivalent

to comparing the two token probabilities because Softmax preserves logit ordering. The gate is passed directly to Eq. (4).

4. Experiments

Following the Security-Efficiency-Utility (SEU) evaluation framework of Wang et al. [19], we evaluate HE-Guardrail along three perspectives: security (attack success rate, pass guardrail rate), efficiency (memory usage and latency overhead), and utility (false rejection of benign prompts). The target responses and guardrail decisions are computed in the encrypted domain.

4.1. Experimental Setup and Metrics

Implementation. We implemented all HE models and guardrails using `desilofhe` [50], a Python-based HE library that supports the RNS-CKKS scheme with GPU acceleration. We set the number of slots as $N/2 = 2^{15}$ and the available multiplicative level after bootstrapping as 17. All experiments were conducted on two NVIDIA H200 GPUs running AlmaLinux 9.8: the target model Llama-3-8B-Instruct occupies one GPU, and a separately scheduled guardrail occupies the other. For the target model, we prioritize numerical accuracy over depth: SiLU is approximated by a degree-127 polynomial, RMSNorm evaluates the inverse square root via Goldschmidt iterations, and Softmax follows Cho et al. [51].

The padded input length is 2,048 tokens, including any guardrail instructions, and the generation budget is 200 tokens. Under multi-model serving, this yields $2^{15}/2,048 = 16$ interleaved lanes. Joint target–Llama Guard execution assigns two lanes per request, so the same layout admits eight guarded requests instead of sixteen target-only requests.

Attack and Benign Inputs. Following Wang et al. [19], we evaluate six jailbreak attack families comprising 2,215 prompts or sessions: IJP [39] (manual), GCG [15] and AutoDAN [16] (optimization-based), DrAttack [42] and MultiJail [43] (implicit), and ActorAttack [44] (multi-turn). For benign-input evaluation, we use AlpacaEval [52] and OR-Bench [53] to measure false-positive rejection.

Metrics and Judge. We judge target responses with `cais/HarmBench-Llama-2-13b-cls` [54]. For an attack sample i , let $a_i \in \{0, 1\}$ indicate guardrail acceptance and $h_i \in \{0, 1\}$ indicate a harmful-response judgment. For an attack set $\mathcal{A}$, attack success rate (ASR) and pass-guardrail rate (PGR) are

$$\text{ASR} = \frac{\sum_{i \in \mathcal{A}} a_i h_i}{|\mathcal{A}|}, \quad \text{PGR} = \frac{\sum_{i \in \mathcal{A}} a_i}{|\mathcal{A}|}. \quad (27)$$

For a benign set $\mathcal{B}$, the false-positive rate is $\text{FPR} = |\mathcal{B}|^{-1} \sum_{i \in \mathcal{B}} (1 - a_i)$. PGR separates prompt rejection from the target’s own refusal behavior: low ASR alone need not indicate an effective guardrail.

TABLE 1. ATTACK SUCCESS RATE (ASR) AND PASS-GUARDRAIL RATE (PGR) IN PERCENT; LOWER IS BETTER. ALL TARGET RESPONSES AND GUARDRAIL DECISIONS ARE COMPUTED IN THE ENCRYPTED DOMAIN, AND HARMFULNESS IS JUDGED BY HARMBENCH-LLAMA-2-13B-CLS [54].

		No defense	Llama Guard 3-8B		JBShield-D		GradSafe-Zero	
Attack	Samples	ASR	ASR	PGR	ASR	PGR	ASR	PGR
<i>Single-turn attacks</i>								
IJP	1,000	14.3	9.6	33.6	14.3	95.4	14.2	59.9
GCG	100	13.0	2.0	3.0	13.0	72.0	13.0	77.0
AutoDAN	100	2.0	0.0	21.0	2.0	46.0	1.0	4.0
DrAttack	100	11.0	7.0	39.0	11.0	52.0	11.0	42.0
MultiJail	315	9.5	7.9	37.1	9.5	97.1	9.5	91.1
<i>Multi-turn attacks</i>								
ActorAttack	600	16.3	16.2	98.5	16.3	99.2	16.2	99.8
Average	–	11.0	7.1	38.7	11.0	77.0	10.8	62.3

4.2. Jailbreak Defense

Across all evaluated prompts, the encrypted guardrails reproduce the plaintext decisions in 99.7% of cases, indicating that the polynomial approximations and HE numerical error rarely alter the safety decision itself. The following comparisons therefore reflect the underlying detection rules rather than HE-induced degradation.

Table 1 reports ASR for the six evaluated attack families. The macro-average ASR is 11.0% without a guardrail, 7.1% for Llama Guard, 11.0% for JBShield, and 10.8% for GradSafe. Llama Guard lowers ASR on every family, most notably from 13.0% to 2.0% on GCG and from 14.3% to 9.6% on IJP, while ActorAttack changes only from 16.3% to 16.2%. JBShield matches the no-defense ASR on all six families, and GradSafe differs by at most 1.0 percentage point.

The average PGR is 38.7% for Llama Guard, 77.0% for JBShield, and 62.3% for GradSafe. All three guardrails pass 98.5–99.8% of ActorAttack sessions. The per-family values vary widely: on GCG, PGR is 3.0% for Llama Guard but 72.0% for JBShield and 77.0% for GradSafe, whereas on AutoDAN it is 21.0%, 46.0%, and 4.0%, respectively.

4.3. Memory Usage and Latency Overhead

GPU Memory Usage. Table 2 reports approximate memory increases relative to running the target model alone. A separately evaluated Llama Guard adds approximately 140 GiB and occupies the second H200 GPU. JBShield adds approximately 5 GiB because its online work uses a hidden-state vector and calibrated references. GradSafe adds approximately 110 GiB: streaming similarities avoid retaining all gradients, but backward computation still requires a substantial checkpoint working set.

Joint target–Llama Guard serving incurs no additional ciphertext memory, since both models share the same interleaved ciphertexts. This comes at the cost of halved batch capacity: the sixteen lanes accommodate eight guarded requests instead of sixteen target-only requests.

TABLE 2. APPROXIMATE INCREMENTAL GPU MEMORY RELATIVE TO TARGET-ONLY INFERENCE.

Configuration	Additional memory (GiB)
Llama Guard: separate execution	≈ 140
Llama Guard: multi-model serving	$\approx 0^*$
JBShield-D	≈ 5
GradSafe-Zero	≈ 110

*At fixed sixteen-lane capacity: eight target–guardrail pairs instead of sixteen target-only requests.

Concurrent Latency. All three guardrails run concurrently with target-model decoding. Llama Guard is evaluated alongside the target prefill under multi-model serving, whereas JBShield and GradSafe start as soon as the prefill signals they require become available and proceed independently of subsequent decoding. Following the SEU framework [19], let τ_T denote the end-to-end response time of target-only inference and τ_{T+G} that of the target model with a guardrail, both measured from request submission to the release of the final response. The latency overhead of the guardrail is

$$\Delta\tau = \tau_{T+G} - \tau_T, \quad (28)$$

which is zero for every evaluated guardrail in our two-GPU configuration: because the 200-token autoregressive decoding dominates end-to-end latency, each guardrail decision is ready before the final response is released. The guardrail therefore adds no latency on the target model’s critical path, although it still consumes computation and memory on the second GPU.

4.4. Benign-Prompt Utility

Table 3 reports the FPR of each guardrail on benign prompts. Llama Guard rejects 33 of 1,805 benign prompts (1.83%), compared with one (0.06%) for JBShield and three (0.17%) for GradSafe. On OR-Bench, which is designed to probe over-refusal, Llama Guard rejects 22 of 1,000 prompts. Its stronger attack rejection therefore comes at the cost of higher benign rejection, whereas the low FPRs

TABLE 3. FALSE-POSITIVE RATE (%) ON BENIGN PROMPTS, WITH THE NUMBER OF REJECTED PROMPTS IN PARENTHESES.

Guardrail	AlpacaEval ($n = 805$)	OR-Bench ($n = 1,000$)	Combined ($n = 1,805$)
Llama Guard	1.37 (11)	2.20 (22)	1.83 (33)
JBSshield-D	0.12 (1)	0.00 (0)	0.06 (1)
GradSafe-Zero	0.12 (1)	0.20 (2)	0.17 (3)

of JBSshield and GradSafe reflect their more permissive detection rules rather than uniformly better guardrails.

5. Future Work

Multi-turn jailbreak attacks remain an important limitation of current guardrail mechanisms. As reported by Wang et al. [19], even session-level guardrails exhibit substantial vulnerability to multi-turn attacks, particularly against adaptive attacks such as X-Teaming [45]. This limitation arises because the malicious intent may emerge gradually across multiple interactions rather than being identifiable from an individual prompt or response.

In plaintext LLM deployments, conversation histories can at least be directly inspected by service operators or subjected to human moderation as an operational fallback. In HE-LLM inference, however, the server cannot directly inspect either the prompts or generated responses, making such plaintext-level intervention unavailable. Given both the high attack success rates of multi-turn jailbreaks and the lack of plaintext visibility under HE, developing HE-compatible guardrails that securely reason over encrypted conversation histories remains an important direction for future work.

6. Conclusion

In this work, we identified a security vulnerability in HE-LLM inference: the confidentiality that protects benign clients also allows malicious clients to submit jailbreak prompts that the server can neither inspect nor block. To address this vulnerability, we proposed HE-Guardrail, a framework that evaluates jailbreak guardrails entirely over encrypted data and enforces their decisions through homomorphic response gating with gated noise flooding, so that the suppressed response cannot be recovered by the client. We instantiated HE-Guardrail with Llama Guard, JBSshield, and GradSafe, covering both pre-processing and intra-processing guardrails, and developed the HE-compatible building blocks they require, including encrypted gradient evaluation for GradSafe and multi-model serving for concurrent evaluation of the target and guardrail models. Our evaluation shows that the encrypted guardrails achieve defense performance close to that of their plaintext counterparts, with distinct security–efficiency–utility trade-offs across guardrail designs, and that guardrail evaluation can be overlapped with target generation without adding latency to the response.

More broadly, our threat model departs from the conventional assumption in HE-based PPML that the client is benign in its choice of input. Considering a client that follows the protocol yet adversarially exploits the encrypted model is a necessary step toward trustworthy HE-based inference, in which the model itself must be protected in addition to the client’s data. Overall, these results bring HE-LLM inference closer to practical deployment and open a direction for further research on safety mechanisms for encrypted LLM inference.

References

- [1] M. Al-Rubaie and J. M. Chang, “Privacy-preserving machine learning: threats and solutions,” *IEEE Security Privacy*, vol. 17, no. 2, pp. 49–58, Mar. 2019.
- [2] M. S. Riaz, B. D. Rouani, and F. Koushanfar, “Deep learning on private data,” *IEEE Security Privacy*, vol. 17, no. 6, pp. 54–63, Nov. 2019.
- [3] R. Xu, N. Baracaldo, and J. Joshi, “Privacy-preserving machine learning: methods, challenges and directions,” *arXiv preprint arXiv:2108.04417*, Sep. 2021.
- [4] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2016, pp. 308–318.
- [5] P. Mohassel and Y. Zhang, “SecureML: a system for scalable privacy-preserving machine learning,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 19–38.
- [6] P. Mohassel and P. Rindal, “ABY3: a mixed protocol framework for machine learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2018, pp. 35–52.
- [7] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, “CryptoNets: applying neural networks to encrypted data with high throughput and accuracy,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jun. 2016, pp. 201–210.
- [8] E. Lee, J.-W. Lee, J. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and W. Choi, “Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2022, pp. 12 403–12 422.
- [9] J. Moon, D. Yoo, X. Jiang, and M. Kim, “THOR: secure transformer inference with homomorphic encryption,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2025, pp. 3765–3779.
- [10] D. Park, E. Lee, and J.-W. Lee, “Powerformer: efficient and high-accuracy privacy-preserving language model with homomorphic encryption,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Jul. 2025, pp. 11 090–11 111.
- [11] D. Rho, T. Kim, M. Park, J. W. Kim, H. Chae, E. K. Ryu, and J. H. Cheon, “Encryption-friendly LLM architecture,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, Apr. 2025, pp. 67 315–67 341.
- [12] L. D. Castro, D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, “EncryptedLLM: privacy-preserving large language model inference via GPU-accelerated fully homomorphic encryption,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2025.
- [13] L. Zhang, X. Wang, J. J. Sim, Z. Huang, J. Zhong, H. Wang, P. Duan, and K. Y. Lam, “MOAI: module-optimizing architecture for non-interactive secure transformer inference,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, Apr. 2026.
- [14] Y. Yu, Y. Zhou, Y. Chen, P. Soto, W. Xiong, and M. Li, “Cachemir: fully homomorphic encrypted inference of generative large language model with kv cache,” *arXiv preprint*, Feb. 2026, arXiv:2602.11470.

- [15] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv:2307.15043, Jul. 2023.
- [16] X. Liu, N. Xu, M. Chen, and C. Xiao, "Autodan: generating stealthy jailbreak prompts on aligned large language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, May 2024, pp. 56 174–56 194.
- [17] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "Nemo guardrails: a toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, Dec. 2023, pp. 431–445.
- [18] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine *et al.*, "Llama guard: LLM-based input-output safeguard for human-AI conversations," arXiv preprint, Dec. 2023, arXiv:2312.06674.
- [19] X. Wang, Z. Ji, W. Wang, Z. Li, D. Wu, and S. Wang, "SoK: evaluating jailbreak guardrails for large language models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2026, pp. 39–58.
- [20] X. Wang, D. Wu, Z. Ji, Z. Li, P. Ma, S. Wang, Y. Li, Y. Liu, N. Liu, and J. Rahmel, "SelfDefend: LLMs can defend themselves against jailbreaking in a practical manner," in *Proc. USENIX Secur. Symp.*, Aug. 2025, pp. 2441–2460.
- [21] Y. Xie, M. Fang, R. Pi, and N. Gong, "Gradsafe: detecting jailbreak prompts for LLMs via safety-critical gradient analysis," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Aug. 2024, pp. 507–518.
- [22] S. Zhang, Y. Zhai, K. Guo, H. Hu, S. Guo, Z. Fang, L. Zhao, C. Shen, C. Wang, and Q. Wang, "JBSHield: defending large language models from jailbreak attacks through activated concept analysis and manipulation," in *Proc. USENIX Secur. Symp.*, Aug. 2025, pp. 8215–8234.
- [23] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "Smooth-llm: defending large language models against jailbreaking attacks," arXiv:2310.03684, Oct. 2023.
- [24] X. Zhang, C. Zhang, T. Li, Y. Huang, X. Jia, M. Hu, J. Zhang, Y. Liu, S. Ma, and C. Shen, "Jailguard: a universal detection framework for prompt-based attacks on LLM systems," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 1, pp. 1–40, 2025.
- [25] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A full rns variant of approximate homomorphic encryption," in *Proc. Int. Conf. Sel. Areas Cryptogr. (SAC)*, Aug. 2018, pp. 347–368.
- [26] C. Gentry and Y. Lee, "Fully homomorphic encryption for matrix arithmetic," in *Proc. Annu. Int. Cryptol. Conf. (CRYPTO)*, Aug. 2026, pp. 363–395.
- [27] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur. (ASIACRYPT)*, Dec. 2017, pp. 409–437.
- [28] B. Li and D. Micciancio, "On the security of homomorphic encryption on approximate numbers," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn. (EUROCRYPT)*, Oct. 2021, pp. 648–677.
- [29] W. Ao and V. N. Boddeti, "AutoFHE: automated adaption of CNNs for efficient evaluation over FHE," in *Proc. USENIX Secur. Symp.*, Aug. 2024, pp. 2173–2190.
- [30] J. Zhang, X. Yang, L. He, K. Chen, W. jie Lu, Y. Wang, X. Hou, J. Liu, K. Ren, and X. Yang, "Secure transformer inference made non-interactive," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, Feb. 2025.
- [31] L. Lim, V. Kalagi, D. Agrawal, and A. E. Abbadi, "Tricycle: private transformer inference with tricyclic encodings," Cryptology ePrint Archive, Paper 2025/1200, Jun. 2025. [Online]. Available: <https://eprint.iacr.org/2025/1200>
- [32] L. Yang, J. Chen, W. Dai, S. Wang, W. Wu, and Y. Feng, "ARION: attention-optimized transformer inference on encrypted data," Cryptology ePrint Archive, Paper 2025/2271, Dec. 2025. [Online]. Available: <https://eprint.iacr.org/2025/2271>
- [33] S. Jayashankar, J. Kim, M. B. Sullivan, W. Zheng, and D. Skarlatos, "A scalable multi-GPU framework for encrypted large-model inference," arXiv:2512.11269, Dec. 2025.
- [34] Z. Wang, R. Chen, X. Gao, Y. Wang, L. Liu, Z. Lan, Z. Wang, S. Fu, Q. Wang, and X. Huang, "STIP: efficient and secure non-interactive transformer inference via compact packing," Cryptology ePrint Archive, Paper 2026/174, Feb. 2026. [Online]. Available: <https://eprint.iacr.org/2026/174>
- [35] X. Gao, S. Fu, L. Liu, Z. Liu, Y. Luo, and Y. Wang, "Euston: efficient and user-friendly secure transformer inference with non-interactivity," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2026, pp. 1443–1462.
- [36] J. Park, S. Park, J. H. Park, J. H. Ahn, J. H. Cheon, G. Hanrot, J. W. Kim, M. Park, and D. Stehlé, "Scaling up FHE-based privacy-preserving ML: higher throughput, longer inputs for Llama-3-8B," arXiv:2601.18511, Jun. 2026.
- [37] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: private inference on transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, Nov. 2022, pp. 15 718–15 731.
- [38] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: a low latency framework for secure neural network inference," in *Proc. USENIX Secur. Symp.*, Aug. 2018, pp. 1651–1669.
- [39] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "Do anything now": characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2024, pp. 1671–1685.
- [40] A. Mehrotra, M. Zampetakis, P. Kadian, B. Nelson, H. Anderson, Y. Singer, and A. Karbasi, "Tree of attacks: jailbreaking black-box LLMs automatically," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, Dec. 2024, pp. 61 065–61 105.
- [41] J. Yu, X. Lin, Z. Yu, and X. Xing, "LLM-Fuzzer: scaling assessment of large language model jailbreaks," in *Proc. USENIX Secur. Symp.*, Aug. 2024, pp. 4657–4674.
- [42] X. Li, R. Wang, M. Cheng, T. Zhou, and C.-J. Hsieh, "DrAttack: prompt decomposition and reconstruction makes powerful LLMs jailbreakers," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, Nov. 2024, pp. 13 891–13 913, findings.
- [43] Y. Deng, W. Zhang, S. J. Pan, and L. Bing, "Multilingual jailbreak challenges in large language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, May 2024, pp. 24 634–24 651.
- [44] Q. Ren, H. Li, D. Liu, Z. Xie, X. Lu, Y. Qiao, L. Sha, J. Yan, L. Ma, and J. Shao, "LLMs know their vulnerabilities: uncover safety gaps through natural distribution shifts," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Jul. 2025, pp. 24 763–24 785.
- [45] S. Rahman, L. Jiang, J. Shiffer, G. Liu, S. Issaka, M. R. Parvez, H. Palangi, K.-W. Chang, Y. Choi, and S. Gabriel, "X-Teaming: multi-turn jailbreaks and defenses with adaptive multi-agents," arXiv:2504.13203, Apr. 2025.
- [46] Y. Liu, H. Gao, S. Zhai, Y. He, J. Xia, Z. Hu, Y. Chen, X. Yang, J. Zhang, S. Z. Li, H. Xiong, and B. Hooi, "GuardReasoner: towards reasoning-based LLM safeguards," arXiv:2501.18492, Jan. 2025.
- [47] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, Jul. 2024.
- [48] E. Lee, J.-W. Lee, J.-S. No, and Y.-S. Kim, "Minimax approximation of sign function by composite polynomial for homomorphic comparison," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3711–3727, Nov.-Dec. 2022.
- [49] P. Panzade, J. R. Asl, D. Takabi, and Z. Cai, "BlindTuner: on enhancement of privacy-preserving fine-tuning of transformers based on homomorphic encryption," *IEEE Internet Things J.*, vol. 12, no. 19, pp. 39 211–39 222, Oct. 2025.
- [50] DESILO Inc., "DESILO FHE library," 2025. [Online]. Available: <https://desilo.ai>

- [51] W. Cho, G. Hanrot, T. Kim, M. Park, and D. Stehlé, “Fast and accurate homomorphic softmax evaluation,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2024, pp. 4391–4404.
- [52] X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, and T. B. Hashimoto, “AlpacaEval: an automatic evaluator of instruction-following models,” GitHub repository, May 2023. [Online]. Available: https://github.com/tatsu-lab/alpaca_eval
- [53] J. Cui, W.-L. Chiang, I. Stoica, and C.-J. Hsieh, “OR-Bench: an over-refusal benchmark for large language models,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2025, pp. 11 515–11 542.
- [54] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks, “HarmBench: a standardized evaluation framework for automated red teaming and robust refusal,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2024, pp. 35 181–35 224.